FACULTY OF COMPUTING

DEPARTMENT OF INFORMATION SYSTEMS ENGINEERING — FACULTY OF COMPUTING

ASSIGNMENT

Module / Unit	IE3010 — Network Programming (Year 3, Semester 2)
Assignment Title	NetMessenger: A Multi-Client Chat and File-Sharing Platform over TCP/IP
Learning Outcomes Covered	LO5: Design and implement client/server programs using socket communication.
Assignment Mode	Individual. All students complete the same Part 1 specification below, personalised to their own registration number (2.4). The Lab Assessment and Viva are individual, conducted in person in a supervised lab session.
Maximum Marks	100 (Part 1: 40 · Lab Assessment: 30 · Viva: 30)
Contribution to Final Grade	25% of the overall module grade (replaces the previous single Assignment component in the module outline).
Date Published	[Insert date — Week 1 of semester]
Deadline for Submissions	Part 1 (take-home) — ALL materials (GitHub repository link, ZIP archive, Implementation Report, prompt log, reflection): 30th September 2026, 11:59 PM. This is a hard deadline for every artefact, not only the CourseWeb upload. GitHub commits: only commits made to your repository on or before 30th September 2026, 11:59 PM will be considered. Any commits made after this date/time will be disregarded when assessing your process evidence, even if your repository link was already submitted. Lab Assessment + Viva: conducted in a scheduled lab session in Weeks 9–10 (after the Part 1 deadline). Exact date/time and sign-up details will be posted on CourseWeb.
Mode of Submission	Part 1: GitHub repository link + a ZIP archive named IE3010_<your registration number>.zip (2.4) + Implementation Report (PDF/Word, with code and output screenshots) + prompt log + reflection, all uploaded via CourseWeb by the deadline.check section 9 for all required documents. Lab Assessment + Viva: conducted in person on department lab machines under examiner supervision.

1. Overview

This assignment has three assessed components, all based on the same piece of work.

- Part 1 — Take-Home Implementation (40%): you design and build, at home, a working client–server chat and file-sharing application to the specification below, personalised to your own registration number. This is the same task for every student, but every submission is individually identifiable.

- Lab Assessment (30%): a supervised, in-lab, closed-book practical test in which you extend or modify your own Part 1 implementation. Full details of what will be tested are given in 5 — the specific task is only revealed to you in the lab.
- Viva (30%): a short individual oral examination, conducted around the same lab session, on your Part 1 design and the modification you make in the Lab Assessment.

You may use AI tools to help with Part 1 (see 3). The Lab Assessment and Viva are done entirely without AI, notes, or internet access — you are expected to understand your own submission well enough to extend it live and explain it to an examiner. Being unable to explain or build on your own Part 1 code is treated as evidence that the work submitted is not authentically yours.

2. Part 1 — Take-Home Implementation (40 marks)

2.1 Scenario

You are to implement "NetMessenger": a multi-client chat and file-sharing platform consisting of one server program and one client program, communicating over TCP/IP. The communication protocol is fully specified for you in 2.3 below — your task is to implement it correctly and robustly, not to design a new one. Several details of your specific implementation (listening port, file names, a response tag, your log file, and where files are stored) are personalised to your own registration number — see 2.4.

2.2 Mandatory functional requirements

1. Connection & concurrency: the server must accept and correctly serve multiple clients connected at the same time (via threads, forked processes, or I/O multiplexing — select/poll/epoll). Justify your choice of concurrency model in your Implementation Report.

2. Registration & presence: on connecting, a client registers a unique username. The server maintains a list of currently connected users and notifies all clients when a user joins or leaves.

3. Broadcast messaging: any client can send a message that is delivered to all other connected clients.

4. Private messaging: a client can send a message to a single named client, delivered only to that client.

5. Chat rooms/channels: clients can create a room, join an existing room, leave a room, and list currently available rooms. Messages sent "to a room" are delivered only to members of that room.

6. File sharing: a client can send a file (via the server) to another named client or to a room, using the SENDFILE command in 2.3. The receiving client(s) must receive the complete, unmodified file, and the server must store a copy under your personalised storage path (2.4).

7. Graceful disconnect: the server must detect client disconnects (including ungraceful ones, e.g. the client process being killed) and clean up associated state (remove from user list and any rooms) without crashing.

8. Error handling: the system must not crash on malformed input, unexpected disconnects, or invalid commands (e.g. sending to a non-existent user or room). Define and return sensible error responses.

9. Server-side logging: the server logs connections, disconnections, and message/file-transfer events with timestamps, to your personalised log file (2.4).

2.3 Communication Protocol (given — implement exactly as specified)

Do not design your own protocol. NetMessenger uses the fixed, line-based text protocol defined below. Every command and response is a single line terminated by a newline (\n), except SENDFILE, which is followed immediately by the raw file bytes (see the table). Implement it exactly as given — you will lose marks for inventing your own message formats or deviating from this specification.

Command (client → server)	Format	Server response	Notes
Register	REGISTER <username>	OK REGISTERED <username> ERR 001 USERNAME_TAKEN	Must be the first command on a new connection.
List users	LIST	OK USERS <comma-separated usernames>	Currently connected, registered users.
Broadcast	BCAST <message>	OK SENT (to sender)	All other clients receive: MSG BCAST <sender> <message>

Command (client → server)	Format	Server response	Notes
Private message	PMSG <username> <message>	OK SENT ERR 002 USER_NOT_FOUND	Target receives: MSG PRIV <sender> <message>
Join / create room	JOIN <room>	OK JOINED <room>	Creates the room if it does not already exist.
Leave room	LEAVE <room>	OK LEFT <room> ERR 003 ROOM_NOT_FOUND	
List rooms	ROOMS	OK ROOMS <comma-separated room names>	
Room message	RMSG <room> <message>	OK SENT ERR 003 ROOM_NOT_FOUND	Members receive: MSG ROOM <room> <sender> <message>
Send file	SENDFILE <target> <filename> <filesize> (immediately followed by exactly <filesize> raw bytes — no extra newline before or after)	OK FILE_RECEIVED <filename> ERR 004 FILE_TOO_LARGE ERR 002/003 (unknown user/room)	<target> is a username or a room name.
Quit	QUIT	OK BYE	Server then closes the connection cleanly.

- Framing rule: every text command/response is exactly one line ending in \n. The server must correctly handle a single recv() returning a partial line, multiple lines in one buffer, and, for SENDFILE, must read exactly <filesize> bytes regardless of how many recv() calls that takes.
- Error responses always follow the format: ERR <code> <REASON_CODE>, e.g. ERR 002 USER_NOT_FOUND. You may choose your own numeric codes, but reason codes must be self-explanatory and consistent.
- Personalised response tag: every single response your server sends (every OK and every ERR line, but not the MSG lines forwarded to other clients) must end with a space followed by your Node ID tag, NID:<your four digits> — see 2.4 for how to calculate it. For example:

  OK SENT NID:1234 or ERR 002 USER_NOT_FOUND NID:1234.

2.4 Personalisation Requirements (mandatory — unique to your registration number)

The table below defines several implementation details that must be derived from your own SLIIT registration number. These make every submission individually identifiable even though the specification is the same for everyone. Getting a formula wrong, or hard-coding a value that ignores it, will lose marks under "Protocol Implementation, Personalisation & Report" in the Marking Scheme (7), independently of whether the underlying feature otherwise works.

Item	Formula	Worked example (registration number IT21123456)
Server/client listening port	6000 + last four digits of your registration number	Last four digits: 3456 → port = 6000 + 3456 = 9456
Source file names	server_<last4digits>.c, client_<last4digits>.c, Makefile_<last4digits>	server_3456.c, client_3456.c, Makefile_3456
Node ID (NID) tag	Middle four digits of the 8-digit numeric part of your registration number (digits 3–6 of that sequence)	Numeric part: 21123456 → middle four digits (3rd–6th): 1234 → NID:1234
Log file name	netmsg_<full registration number>.log	netmsg_IT21123456.log
File storage path	./storage/<full registration number>/<sender_username>/<filename>	./storage/IT21123456/amal/report.pdf
Submission archive	IE3010_<full registration number>.zip	IE3010_IT21123456.zip

- State your full registration number, and every value you calculated from it (port, NID, file names, log name, storage path), clearly at the top of your Implementation Report and in your README.
- If your registration number has a different format from the example (e.g. a different number of digits), use the equivalent last/middle four digits of the numeric part and explain your calculation in your report.

2.5 Constraints

- Implement entirely in C, using the standard BSD sockets API (sys/socket.h and related headers) directly. Do not use a high-level web, chat, or RPC framework or library that hides the socket layer — the point of the exercise is the socket-level work.
- Your code must compile cleanly with gcc on a standard Linux environment (the department lab machines), using your personalised Makefile (2.4).
- A single server process/program must be able to serve at least 5 simultaneous clients on a single machine for testing purposes.

2.6 Optional extensions (for higher marks under Functional Implementation & Correctness)

- A basic security layer, e.g. TLS/SSL via the standard library, or a lightweight authentication/token mechanism (addresses LO4).
- Message/chat history persisted to disk and replayed to a client on reconnect.
- A UDP-based presence/heartbeat channel running alongside the main TCP channel.
- Rate limiting or basic flood protection.

2.7 Implementation Report (required)

Submit a detailed Implementation Report (PDF or Word) documenting your work. This is not a short design note — it must give an examiner who has not seen your screen a clear, evidenced picture of what you built and how it behaves. At minimum it must include:

- Your registration number and every personalised value calculated from it (port, NID, file names, log file name, storage path), stated up front.
- Architecture overview: a diagram or clear description of your client/server architecture and your chosen concurrency model, with justification.
- Protocol implementation evidence: confirmation that each command in the given protocol (2.3) is implemented, with at least one real captured example (e.g. a terminal screenshot) of a full request/response exchange showing your NID tag.
- Personalisation proof: a screenshot showing your server listening on your calculated port (e.g. using ss -tlnp or netstat), a snippet of your correctly-named log file, and a directory listing showing your personalised storage path containing at least one received file.
- Annotated code screenshots: screenshots of the key parts of your source code (connection handling, concurrency, each major message type, error handling, file transfer), each with a short explanation of what it does. Do not paste raw code as plain text in place of screenshots — screenshots must be legible (crop tightly, use a readable font size/zoom in your editor or terminal).
- Execution/output screenshots: screenshots demonstrating the running system — e.g. the server starting up, multiple clients connected simultaneously, broadcast/private/room messaging in action, a successful file transfer, and at least one error case being handled gracefully (e.g. a malformed message or a client disconnecting mid-session).
- Testing summary: a short table or list of the test cases you ran (what you did, expected result, actual result) covering the mandatory features in 2.2.
- Design rationale and assumptions: justify significant design choices and state any assumptions you made where the brief leaves a choice to you.

2.8 Required process evidence

- GitHub repository with a minimum of 8 meaningful, incremental commits with descriptive messages, spread across the assignment period (not all on one day). Only commits timestamped on or before the deadline (see Assignment details, above) count.
- A design diary (0.5–1 page) noting key decisions and any obstacles, updated as you go. This can be used in your viva session.
- A prompt log recording any AI interactions used while building Part 1 — see 3.
- A structured reflection (see 4).

2.9 Deliverables checklist

- Link to GitHub repository (personalised C source files, README with build instructions and your personalised Makefile, commit history, design diary).

- ZIP archive named IE3010_<your registration number>.zip containing your personalised source files (server_<last4>.c, client_<last4>.c), your Makefile_<last4>, a sample excerpt of your log file (netmsg_<regno>.log), and your Implementation Report.
- Implementation Report, as specified in 2.7, including personalisation proof and code/execution screenshots.
- Prompt log / AI interaction record.
- Structured reflection.
- All items above must be uploaded/submitted via CourseWeb by **7th October 2026, 10:59 PM.(NO Extentions)**

3. Usage of AI — Part 1 only (CLEAR Level 3: AI Collaboration)

You may use AI tools to assist with specific tasks in Part 1 — for example, drafting boilerplate socket code, debugging an error message, explaining an API, or getting feedback on your Implementation Report. You must critically evaluate, test, and where necessary modify anything AI produces before it goes into your submission; code you cannot explain should not be in your repository.

- Record all substantive AI interactions (tool used, prompt, and how the output was used or changed) in your prompt log.
- Do not submit AI-generated code, architecture, or prose you do not understand — you will be examined on your submission in the Viva, and being unable to explain your own work will be treated as evidence the work is not authentically yours.
- AI use is not compulsory. Marks are awarded for correct, well-justified, working software, however it was produced.

The Lab Assessment and Viva are CLEAR Level 1: No AI. No AI tools, notes, textbooks, or internet access (beyond any reference material explicitly provided in the lab) are permitted. This is a closed-book, controlled-conditions assessment.

4. Reflection (5 marks, submitted with Part 1)

Submit a short structured reflection (300–500 words) answering:

- Which AI tools, if any, did you use, and at which stages of the work?
- What did the AI do well? Where did it get things wrong or mislead you?
- What did you change, add, or reject from any AI output, and why?
- What did you learn about your own understanding of network programming through completing this assignment?

5. Lab Assessment (30 marks)

5.1 What it is

The Lab Assessment is a supervised, closed-book practical test conducted individually in a department lab. You will extend or modify your own Part 1 implementation to meet a new requirement given to you on the day. The specific requirement is not disclosed in advance and differs from student to student — it is designed to be answerable by anyone who genuinely understands their own Part 1 submission, regardless of which particular change they are asked to make.

5.2 Format and logistics

- Duration: 20 minutes for the modification task.
- Location: department lab, on supervised machines. Bring your student ID.
- You must have submitted Part 1 before your scheduled slot — your own submitted repository will be checked out on the lab machine at the start of your session, and your personalised port/NID will be verified as a quick authenticity check before the task begins.
- Conditions: closed book. No AI tools, no internet access beyond what is explicitly provided, no personal notes, textbooks, or devices.
- Your task is drawn from a bank of modification tasks and revealed only when your session begins. Partial, well-reasoned attempts are marked on their merits — you are not expected to necessarily finish.

5.3 How to prepare

There is no separate content to study for the Lab Assessment beyond genuinely understanding and being able to extend your own Part 1 code: how you implemented the given protocol, your personalisation values, your concurrency model, how your server manages state (users, rooms, connections), and how your code handles errors and disconnects. Students who write their

own well-structured, well-understood implementation for Part 1 — rather than code they cannot explain — are best placed to succeed here.

6. Viva (30 marks)

Following your Lab Assessment slot, you will have a short individual oral examination (approximately 10 minutes) with an examiner. You will be asked to explain and justify design decisions in your Part 1 submission (e.g. how you implemented the given protocol, why your port/NID/log file have the values they do, your concurrency model, how a specific function works, what would happen in a particular edge case) and to explain the modification you attempted in the Lab Assessment. You are expected to be able to point to and explain any part of your own submission, whether or not you used AI assistance while writing it.

7. Marking Scheme

Marks are allocated across all three components, and within Part 1, to both the product you submit and the process by which you produced it.

Criterion	Marks	Excellent	Good	Satisfactory	Poor	Very Poor
PART 1 — TAKE-HOME IMPLEMENTATION (40 marks)						
Functional Implementation & Correctness	15	All mandatory features work correctly; at least one optional extension implemented and working.	All mandatory features work correctly with only minor issues.	Most mandatory features work; some gaps under normal use.	Several mandatory features missing or broken.	Little or none of the required functionality works.
Protocol Implementation, Personalisation & Report	10	Given protocol (§2.3) implemented exactly as specified, including correct framing and file-transfer byte-counting; full personalisation compliance (port, NID tag, file names, log name, storage path all correct); report includes clear, legible code and execution screenshots with a coherent testing summary; architecture cleanly separates concerns.	Protocol implemented correctly for most commands; personalisation applied with only minor errors (e.g. one incorrect file name); report includes adequate screenshots and testing evidence with minor gaps.	Protocol partially implemented or has framing bugs; personalisation only partly applied; report evidence present but thin.	Several protocol commands missing or incorrect; personalisation largely ignored; report lacks meaningful screenshots or testing evidence.	Protocol not implemented as specified; no personalisation applied; report missing or does not evidence the implementation.
Development Process Evidence (commits, design diary, prompt log)	10	Rich, regular, descriptive commit history; diary and prompt log show a clear, honest development story.	Reasonable commit history and diary/prompt log with minor gaps.	Minimal but present evidence of process.	Sparse or last-minute evidence (e.g. one large commit).	No meaningful process evidence provided.
Reflection on Learning & AI Use	5	Specific, honest and analytical; clearly ties AI use (or its absence) to what was learned.	Addresses all four prompts with reasonable specificity.	Addresses the prompts but stays generic.	Superficial or largely templated.	Missing or does not engage with the prompts.
LAB ASSESSMENT — IN-LAB MODIFICATION TEST (30 marks)						

Criterion	Marks	Excellent	Good	Satisfactory	Poor	Very Poor
Live Modification Task — Correctness & Completeness	20	Task completed fully and correctly within the time limit; edge cases considered.	Task mostly completed and functional, minor issues remain.	Partial implementation showing correct understanding of the approach.	Little progress; approach shows misunderstanding.	No meaningful attempt.
Code Quality & Integration Under Time Constraint	10	Change is cleanly integrated into the existing codebase, follows its conventions, and does not break existing functionality.	Change is reasonably well integrated; minor regressions or inconsistencies.	Change works in isolation but is poorly integrated.	Change is bolted on with noticeable regressions.	Change is not integrated / breaks the existing system.
VIVA (30 marks)						
Understanding & Defence of Part 1 Design	15	Confidently and accurately explains all aspects of own protocol implementation and architecture, including edge cases and alternatives considered.	Explains most design decisions correctly with minor prompting.	Explains basic design decisions but struggles with detail.	Struggles to explain own submitted work.	Unable to account for own Part 1 submission.
Understanding & Defence of Lab Modification	15	Clearly explains the reasoning behind the modification and how it interacts with the rest of the system.	Explains the modification correctly with minor gaps.	Gives a basic explanation but misses some detail.	Struggles to explain the change just made.	Cannot explain the modification attempted.

Note: within Part 1, process-related criteria (development process evidence + reflection = 15 of 40 marks) meet the CLEAR Framework's minimum 20% process-mark guidance for AI Collaboration (Level 3) work. The Viva (30 of 100 marks overall) is additional oral process evidence covering the whole assignment.

8. Academic Integrity

Because all students complete the same Part 1 specification, submissions will be checked for code similarity across the cohort in addition to normal plagiarism checks. The personalisation requirements in 2.4 make wholesale copying easy to spot: a submission whose port, NID tag, file names, log name, or storage path do not match the owner's registration number is strong evidence it is not that student's own work. Submitting another student's code, or an unmodified AI output you cannot explain, is not permitted and will be dealt with under the Institute's academic integrity policy. Screenshots in your Implementation Report must be genuinely your own program running — fabricated or edited screenshots will be treated as a serious integrity breach. The Lab Assessment requires live, closed-book work on your own codebase, and the Viva requires you to explain your own design decisions — together these are the primary mechanisms for confirming that your Part 1 submission reflects your own understanding.

9. Deadline Policy

- All Part 1 materials —GitHub repository link, ZIP archive, Implementation Report, prompt log, and reflection) (e.g. IE3010_IT123456789.zip) — are due on CourseWeb by 7th October 2026, 10:59 PM.
- GitHub commit cut-off: only commits made on or before 7th October 2026, 10:59 PM will be considered when evaluating your development process. Commits made after this date/time, even to the same repository, will not be taken into account and may be treated as evidence against the authenticity of your process evidence.
- Standard late-submission penalties apply to submissions made after the deadline, per the module/programme late-submission policy.